

EC 551 – Advanced Digital Design with Verilog and FPGAs

Lecture 4 Introduction to Verilog

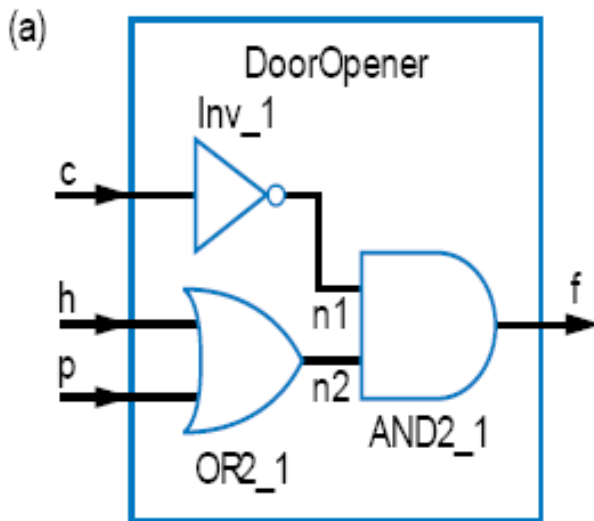
Prof. Douglas Densmore

Announcements

- FPGA tutorial assignment posted to Blackboard.
- HW1 posted to Blackboard. Due 10/1.
- **I will assume you know the material in the FPGA tutorial going forward. Remember to work solo on this. Do NOT have to turn it in.**
- **Project presentation #1 on 10/1**
- **Project presentation material coming soon.**

Why Not English?

- Can describe circuit using English text rather than using a drawing
 - Of course, English isn't a good language for a computer to read
 - Need a more precise, computer-oriented language



- (b) We'll now describe a circuit whose name is DoorOpener.
The external inputs are *c*, *h* and *p*, which are bits.
The external output is *f*, which is a bit.

We assume you know the behavior of these components:

An inverter, which has a bit input *x*, and bit output *F*.

A 2-input OR gate, which has inputs *x* and *y*, and bit output *F*.

A 2-input AND gate, which has bit inputs *x* and *y*, and bit output *F*.

The circuit has internal wires *n1* and *n2*, both bits.

The DoorOpener circuit internally consists of:

An inverter named *Inv_1*, whose input *x* connects to external input *c*, and whose output connects to *n1*.

A 2-input OR gate named *OR2_1*, whose inputs connect to external inputs *h* and *p*, and whose output connects to *n2*.

A 2-input AND gate named *AND2_1*, whose inputs connect to *n1* and *n2*, and whose output connects to external output *f*.

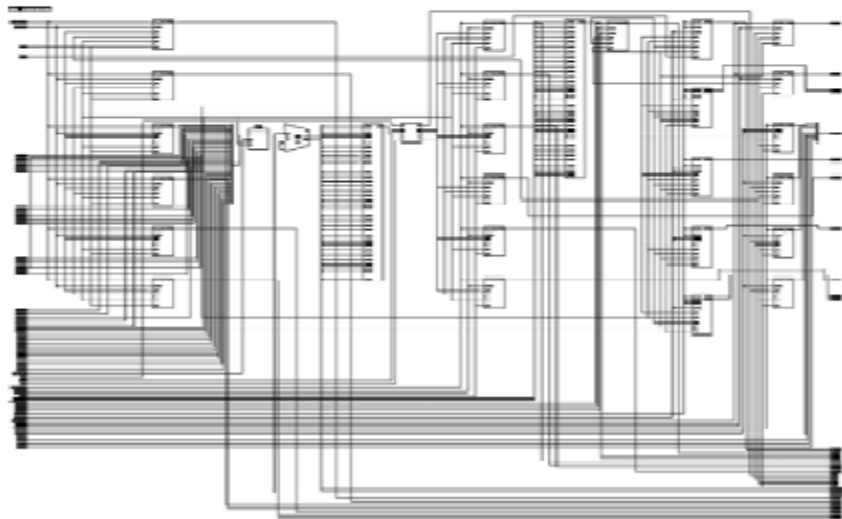
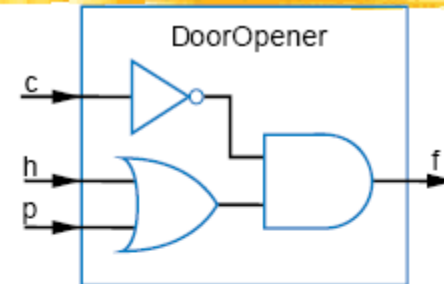
That's all.

Why Not Schematics?

- A drawing of a circuit, or *schematic*, contains graphical information about a design

- Inverter is above the OR gate, AND gate is to the right, etc.

- Such graphical information may not be useful for large designs
- Can use textual language instead



Hardware Description Language (HDL)

- ❑ Hardware description language (HDL): allows designer to **specify** logic function only. Then a **computer-aided design** (CAD) **tool** produces the optimized gates.
- ❑ Most commercial designs built using HDLs
- ❑ Two leading HDLs:
 - **Verilog**
 - developed in 1984 by Gateway Design Automation
 - became an IEEE standard (1364) in 1995
 - **VHDL**
 - Developed in 1981 by the Department of Defense
 - Became an IEEE standard (1076) in 1987

HDL History

- ❑ Many credit Reed for the first HDL
- ❑ **Irving S. Reed, Symbolic synthesis of digital computers**, Proceedings of the 1952 ACM national meeting, pp.90-94
- ❑ The formalism, simply a **list of Boolean functions that define the input to a block of flip-flops driven by a signal clock**
- ❑ **Irving S. Reed** is a mathematician and engineer. He is best known for co-inventing a class of algebraic error-correcting and error-detecting codes known as Reed-Solomon codes in collaboration with Gustave Solomon. He also co-invented the Reed-Muller code.



Irving S. Reed
1923 - 2012

VHDL History

- ❑ **VHSIC** (Very High Speed Integrated Circuit) Hardware Description Language - **VHDL**
- ❑ VHDL (1987) - DoD sponsored standard
 - similar to Ada (emphasis on re-use and maintainability)
 - simulation semantics visible
 - **very general but verbose**
 - IEEE standard
- ❑ Although there are many features absent in Verilog that are unique to VHDL (and vice versa), in practice most of designers use a nearly identical subset **because is what the synthesis tools accept.**
- ❑ **Douglas J. Smith, Hdl Chip Design: A Practical Guide for Designing, Synthesizing & Simulating Asics & Fpgas Using Vhdl or Verilog, Doone Pub, 1998.**

Verilog History

- ❑ ISP (1977) – (instruction set processor) research project at CMU
 - simulation, but no synthesis
- ❑ Abel (1983) - developed by Data-I/O
 - targeted to programmable logic devices
 - not good for much more than state machines
- ❑ Verilog - developed by Gateway (absorbed by Cadence) by Phil Moorby in 1983-1984
 - similar to Pascal and C
 - **delays is only interaction with simulator**
 - fairly efficient and easy to write
 - IEEE standard
- ❑ Cadence bought Gateway in 1989

“Phil's work had as much impact on hardware design as the C-language had on software development in 1970”



VHDL vs. Verilog

❑ Which is “better” Verilog or VHDL?

- Both are adequate for our purposes...
- What you use in industry may be dictated by company preference or government requirement.
- VHDL may be more powerful but very rigid.
 - Very Hard and Difficult to Learn?
 - For beginners, do you want to spend your time in logic design, or fighting the language?
- Verilog may be easier, but you can hang yourself if you are not careful...



Structure vs. Function (Behavior)

- ❑ **Structural description:** - A connection of components.
 - textual replacement for schematic
 - **hierarchical composition** of modules from primitives
- ❑ **Functional description:** describe **what module does**, not how synthesis generates circuit for module
 - A set of Boolean formulas, state transitions, etc.
- ❑ **Simulation description:**
 - A program designed for simulation.
- ❑ Descriptions can **be executed**
- ❑ Major languages:
 - Verilog.
 - VHDL.

Structure vs. Function (Behavior)

- ❑ the module describes a component in the circuit
- ❑ Two ways to describe:
 - Structural Verilog
 - List of components and how they are connected
 - Just like schematics, but using text
 - A net list
 - tedious to write, hard to decode
 - Essential without integrated design tools
 - Behavioral Verilog
 - Describe *what* a component does, not *how* it does it
 - Synthesized into a circuit that has this behavior
 - Result is only as good as the tools
- ❑ Build up a hierarchy of modules

Structure vs. Function (Behavior)

□ Basic Idea:

- Language constructs describe circuit structure
- *Structural descriptions* similar to hierarchical netlist.
- *Behavioral descriptions* use higher-level constructs (similar to conventional programming).

□ Originally designed to help in abstraction and simulation.

- Now “logic synthesis” tools exist to automatically convert from behavioral descriptions to gate netlist.
- This may lead you to **falsely believe** that hardware design can be reduced to writing programs!

□ “Structural” example:

```
Decoder(output x0,x1,x2,x3;
        inputs a,b)
{
    wire abar, bbar;
    inv(bbar, b);
    inv(abar, a);
    nand(x0, abar, bbar);
    nand(x1, abar, b   );
    nand(x2, a,      bbar);
    nand(x3, a,      b   );
}
```

□ “Behavioral” example:

```
Decoder(output x0,x1,x2,x3;
        inputs a,b)
{
    case [a b]
        00: [x0 x1 x2 x3] = 0x1;
        01: [x0 x1 x2 x3] = 0x2;
        10: [x0 x1 x2 x3] = 0x4;
        11: [x0 x1 x2 x3] = 0x8;
    endcase;
}
```

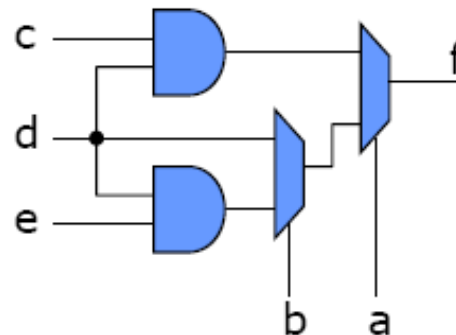
Describing Hardware!!!

- All hardware created during synthesis
 - Even if **a** is true, **still computing** **d&e**
- Learn to understand how descriptions translated to hardware

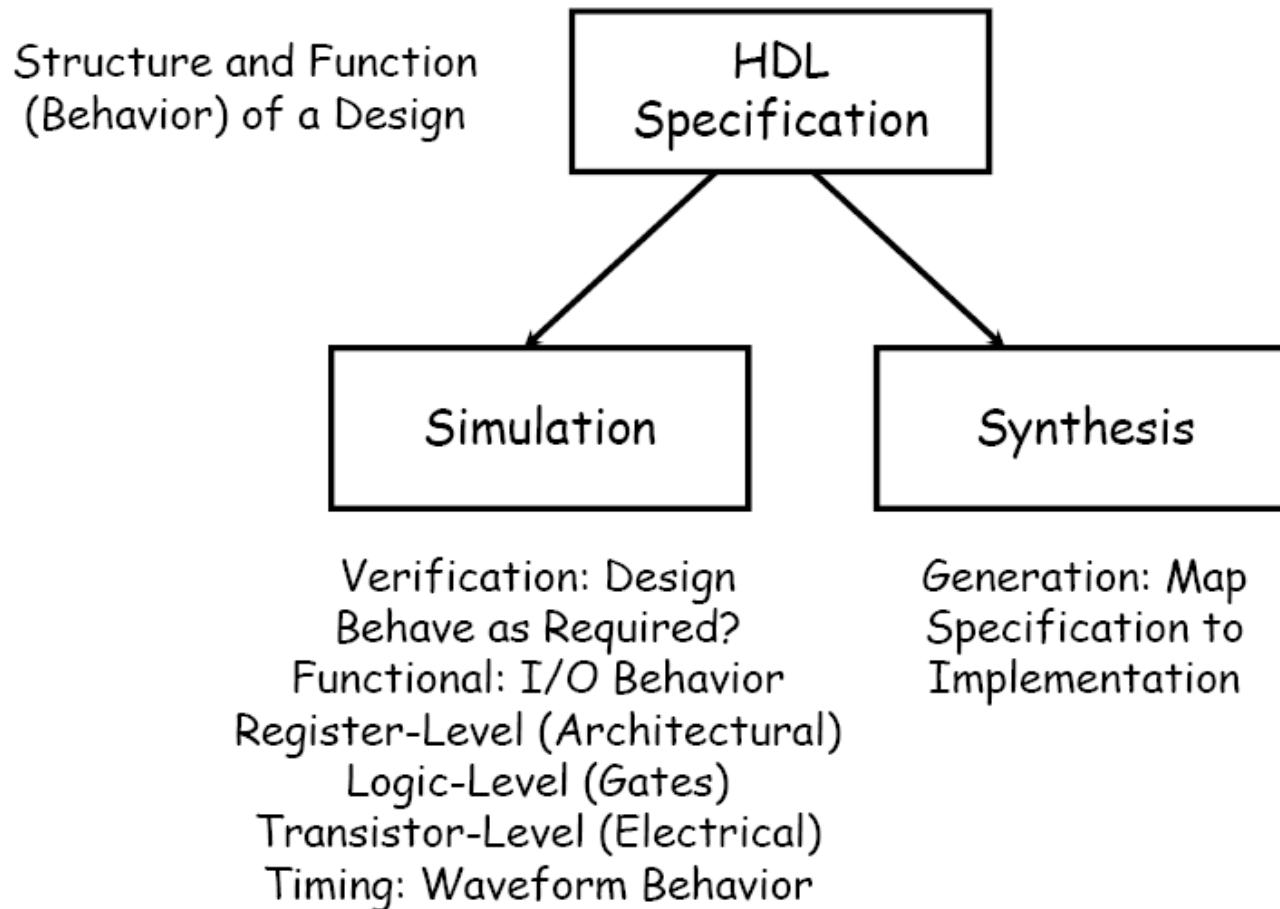
IMPORTANT:

When describing circuits using an HDL, it's critical to think of the **hardware** the code should produce

```
if (a) f = c & d;  
else if (b) f = d;  
else f = d & e;
```



Simulation vs. Synthesis

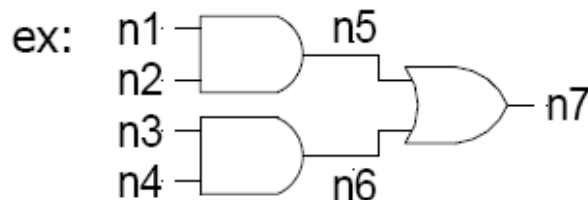


Netlist

- A key data structure (or representation) in the design process is the "netlist":

- Network List

- Netlist lists components and connects them with nodes:



```
g1 "and" n1 n2 n5
g2 "and" n3 n4 n6
g3 "or" n5 n6 n7
```

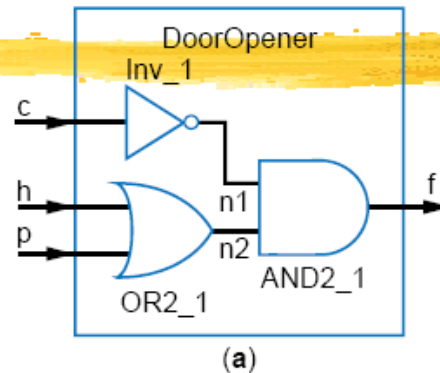
Alternative format:

```
n1 g1.in1
n2 g1.in2
n3 g2.in1
n4 g2.in2
n5 g1.out g3.in1
n6 g2.out g3.in2
n7 g3.out
g1 "and"
g2 "and"
g3 "or"
```

- Netlist is what is needed for simulation and implementation.
- Could be at the transistor level, gate level, ...
- Could be hierarchical or flat.
- How do we generate a netlist?

Describing Structure in Verilog

- ❑ Modules defined for Inv, OR2, and AND2 (details omitted)
 - Note "//" comment
- ❑ Module defined for DoorOpener



- Lists inputs and outputs
- Declares internal wires
- Instantiates and connects three components

We'll now describe a circuit whose name is DoorOpener.
The external inputs are c, h and p, which are bits.
The external output is f, which is a bit.

We assume you know the behavior of these components:
An inverter, which has a bit input x, and bit output F.
A 2-input OR gate, which has inputs x and y, and bit output F.
A 2-input AND gate, which has bit inputs x and y, and bit output F.

The circuit has internal wires n1 and n2, both bits.

The DoorOpener circuit internally consists of:

An inverter named Inv_1, whose input x connects to external input c, and whose output connects to n1.
A 2-input OR gate named OR2_1, whose inputs connect to external inputs h and p, and whose output connects to n2.
A 2-input AND gate named AND2_1, whose inputs connect to n1 and n2, and whose output connects to external output f.

That's all.

```

module Inv(x, F);
    input x;
    output F;
    // details not shown
endmodule

module OR2(x, y, F);
    input x, y;
    output F;
    // details not shown
endmodule

module AND2(x, y, F);
    input x, y;
    output F;
    // details not shown
endmodule

module DoorOpener(c, h, p, f);
    input c, h, p;
    output f;
    wire n1, n2;
    Inv Inv_1(c, n1);
    OR2 OR2_1(h, p, n2);
    AND2 AND2_1(n1, n2, f);
endmodule
    
```

Hierarchy

- ❑ Hierarchical design using *instantiation* of *modules*
- ❑ A module, **like** a procedure in a C-like language, has a name, input/output ports, and a body
- ❑ **A module is not called like a procedure. It is instantiated.**
 - Inputs are monitored. Upon change, the output is changed.
 - Can be done continuously, or under some conditions
 - Output can be instantaneous, or after some delay
 - The module stays around for the lifetime of the program

IMPORTANT:

When describing circuits using an HDL, it's critical to think of the hardware the code should produce

Verilog 101

Required key word Name Port list (wires)

```
module and_gate (out, in1, in2);  
  input          in1, in2;  
  output         out;  
  
  <module internals>  
  
endmodule
```

Verilog Module

- ❑ Encapsulate structural and functional details in a module

```
module my_design (module_ports);
```

```
... // Declarations of ports go here
```

```
... // Structural and functional details go here
```

```
endmodule
```

- ❑ Encapsulation makes the model available for instantiation in other modules

Verilog Primitives

Verilog has 26 built-in primitives (combinational)

n-Input	n-Output, 3-state
and	buf
nand	not
or	bufif0
nor	bufif1
xor	notif0
xnor	notif0

MODELING TIP

The output port of a primitive must be first in the list of ports.
The instance name of a primitive is optional.

Verilog Syntax

- ❑ Verilog is a case sensitive language (with a few exceptions)
- ❑ Identifiers (space-free sequence of symbols)
 - upper and lower case letters from the alphabet
 - digits (*0, 1, ..., 9*)
 - underscore (`_`)
 - `$` symbol (only for system tasks and functions)
 - Max length of 1024 symbols
- ❑ Terminate lines with semicolon
- ❑ Single line comments: `// A single-line comment goes here`
- ❑ Multi-line comments:
`/* Do not /* nest multi-line comments*/ like this */`

Verilog Operators

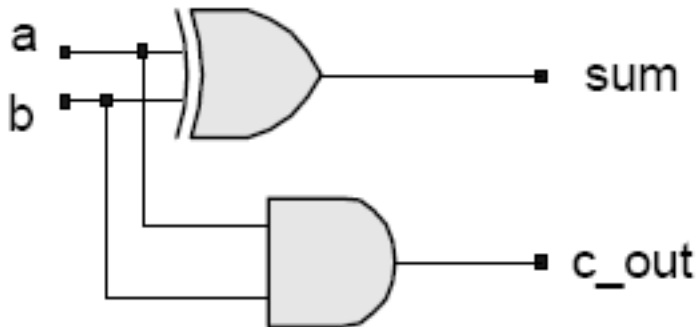
Verilog Operator	Name	Functional Group
[]	bit-select or part-select	
()	parenthesis	
! ~ & ~& ~ ^ ~^ or ^~	logical negation negation reduction AND reduction OR reduction NAND reduction NOR reduction XOR reduction XNOR	Logical Bit-wise Reduction Reduction Reduction Reduction Reduction Reduction
+ -	unary (sign) plus unary (sign) minus	Arithmetic Arithmetic
{ }	concatenation	Concatenation
{ { } }	replication	Replication
* / %	multiply divide modulus	Arithmetic Arithmetic Arithmetic
+ -	binary plus binary minus	Arithmetic Arithmetic
<< >>	shift left shift right	Shift Shift

> >= < <=	greater than greater than or equal to less than less than or equal to	Relational Relational Relational Relational
== !=	logical equality logical inequality	Equality Equality
=== !==	case equality case inequality	Equality Equality
&	bit-wise AND	Bit-wise
^ ^~ or ~^	bit-wise XOR bit-wise XNOR	Bit-wise Bit-wise
	bit-wise OR	Bit-wise
&&	logical AND	Logical
	logical OR	Logical
?:	conditional	Conditional

What is “reduction”?

My first design

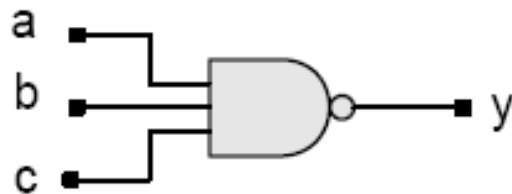
- ❑ Verilog primitives **encapsulate** pre-defined functionality of common logic gates
- ❑ The counterpart of a schematic is a structural model composed of **Verilog primitives**



```
module Add_half (sum, c_out, a, b);  
  input          a, b;  
  output         c_out, sum;  
  xor            (sum, a, b);  
  and            (c_out, a, b);  
endmodule
```

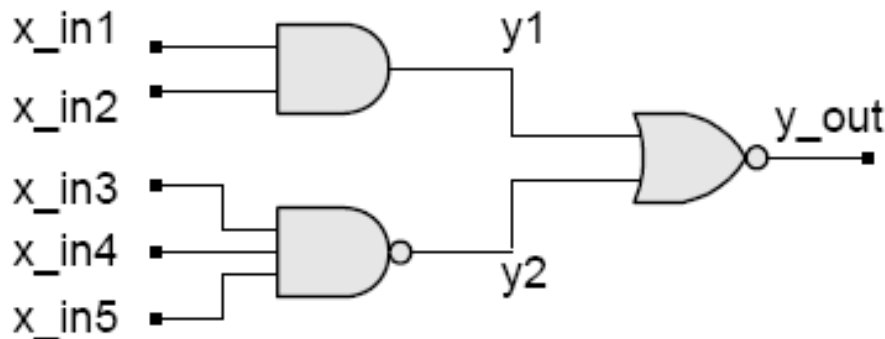
Multi-Input Primitives

Model structural detail by instantiating and connecting primitives



```
...  
nand (y, a, b, c);  
...
```

Structural Details:



```
wire y1, y2;  
nor (y_out, y1, y2);  
and (y1, x_in1, x_in2);  
nand (y2, x_in3, x_in4, x_in5);
```

Structural Model - XOR

```
module xor_gate (out, a, b);  
  input  a, b;  
  output out;  
  wire  abar, bbar, t1, t2;  
endmodule
```

module name

port list

declarations

Built-in gates

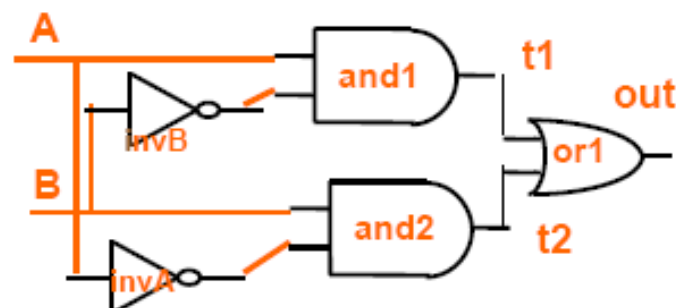
```
inverter invA (abar, a);  
inverter invB (bbar, b);  
and_gate and1 (t1, a, bbar);  
and_gate and2 (t2, b, abar);  
or_gate or1 (out, t1, t2);
```

endmodule

Instance name

interconnections

statements



➤ Composition of primitive gates to form more complex module

➤ Note use of `wire` declaration!

By default, identifiers are wires

Connecting Modules

- ❑ Connect modules together to form larger modules
- ❑ Example:
 - Build an AND gate by 2 NAND gates in series

```
module NAND2 (in1, in2, out);  
    input in1, in2;  
    output out;  
    assign out = ~(in1 & in2);  
endmodule
```

```
module AND2(in1,in2,out);  
    input in1,in2;  
    output out;  
    wire w1; // to connect the 2 NAND's  
    NAND2 nand1(in1,in2,w1);  
    NAND2 nand2(w1,w1,out);  
endmodule
```

Wires

- ❑ Wires in Verilog establish connectivity between primitives and/or modules
- ❑ Data type: nets (Example: **wire**)
- ❑ The logic value of a **wire** (net) is determined dynamically during simulation by what is connected to the wire.

MODELING TIP

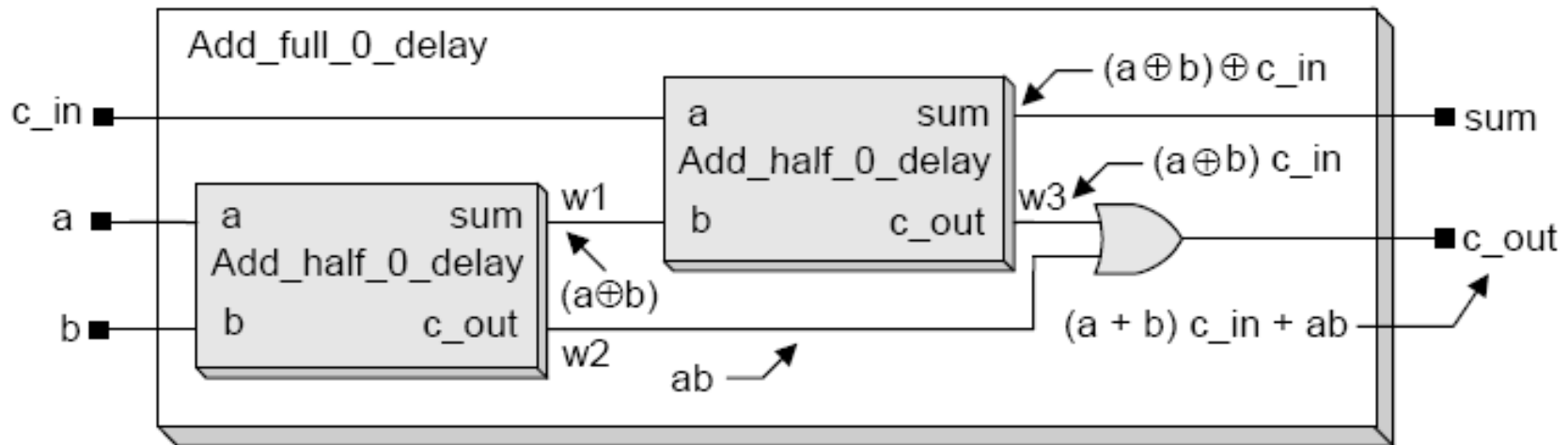
Use nets to establish structural connectivity.

MODELING TIP

An undeclared identifier is treated by default as a **wire**

More Hierarchy

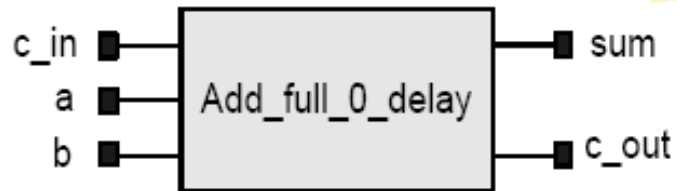
- Model complex structural detail by instantiating modules within modules



MODELING TIP

Use nested module instantiations to create a top-down design hierarchy.

More Hierarchy Again



```
module Add_full_0_delay (sum, c_out, a, b, c_in);  
  input  a, b, c_in;  
  output c_out, sum;  
  wire   w1, w2, w3;  
  
  Add_half_0_delay M1 (w1, w2, a, b);  
  Add_half_0_delay M2 (sum, w3, c_in, w1);  
  or (c_out, w2, w3);  
endmodule
```

module instance
name

MODELING TIP

The ports of a module may be listed in any order.
The instance name of a module is required.

Ports

Port Connection By Name

- The actual ports were associated with the formal posts by their positions in the port list
- Alternatively, connect ports by name in module that have several ports: *.formal_name(actual_name)*
- The first (M1) instantiation of *Add_half_0_delay* in *Add_full_0_delay* can be written:

```
module Add_full_0_delay(sum, c_out, a, b, c_in);  
  output sum, c_out;  
  input a, b, c_in;  
  wire w1, w2, w3;  
  Add_half_0_delay M1 (w1, w2, a, b);  
  Add_half_0_delay M2 (sum, w3, w2, c_in);  
  or M3 (c_out, w2, w3);  
endmodule
```

```
module Add_half_0_delay (sum, c_out, a, b);  
  output sum, c_out;  
  input a, b;
```

Add_half_0_delay M1

formal name

(.b (b),
 .c_out (w2),
 .a (a),
 .sum (w1)
);

actual name

Verilog Coding Style

- ❑ Don't ever just sit down and "code". Think about what hardware you want to build, how to describe it, and how you should test it.
- ❑ You are not writing a computer program, **you are describing hardware... *Verilog is not C!***
- ❑ Only you know what is in your head. If you need help from others, you need to be able to explain your design -- either verbally, or by **detailed comments in your code.**

MUX Design Example

Next Time

- Simulation and Test

Questions?